

Trabajo académico

Greed Island: Proyecto 2 de Estructura de Datos

Integrantes: Daniel Uribe • Lukas Bustamante • Francisco Alvarado

Universidad de Magallanes

Estructura de Datos

Docente: Jacqueline Aldridge

Punta Arenas

30 de octubre de 2026

Resumen

En este informe presentamos *Greed Island*, un juego de rol por terminal hecho en C, inspirado en el arco de Hunter × Hunter. El jugador crea o carga un personaje, explora zonas donde ocurren eventos aleatorios (cartas, trampas y combates), administra un inventario de cartas y puede consultar y deshacer parte de su historial. Para las mecánicas usamos tres estructuras hechas por el grupo: una lista doblemente enlazada con nodo centinela para el inventario, una pila enlazada para el historial y una cola circular para el orden de los enemigos en combate. Explicamos por qué elegimos cada una, cómo funcionan sus operaciones y cómo tratamos los casos límite (vacía, un elemento, sin memoria, eliminación). El programa guarda y carga partidas desde archivo y las estructuras se usan de verdad dentro del juego, no solo se muestran.

Palabras clave: listas enlazadas, pila, cola circular, TDA, memoria dinámica, C, RPG

Índice

1. Introducción **2**

1.1. Objetivos 2

2. Marco teórico **2**

2.1. Manejo de errores 3

3. Metodología **4**

3.1. Organización y desarrollo del proyecto 4

3.2. Implementación y elección de las estructuras de datos 4

3.2.1. Lista doblemente enlazada con nodo centinela: inventario 4

3.2.2. Pila enlazada: historial de acciones 5

3.2.3. Cola circular: orden de enemigos en combate 7

3.3. Funcionamiento del juego 8

3.4. Guardado y carga de partida 8

3.5. Casos límite y manejo de memoria 9

3.6. Declaración de IA 9

3.7. Contribución por estudiante 10

3.8. Procedimiento 10

4. Resultados **11**

5. Discusión **13**

6. Conclusión **14**

1. Introducción

Este proyecto consiste en desarrollar un juego RPG para la asignatura de Estructuras de Datos, implementando listas enlazadas, pilas y colas mediante el uso del lenguaje C.

Nuestro juego está inspirado en el arco de "Greed Island" del anime Hunter × Hunter, donde una de sus mecánicas principales es la recolección de cartas con distintos efectos. En nuestro juego, esta funcionalidad también está presente. Además, el jugador puede explorar distintas zonas, donde se generan ciertos eventos aleatorios, como por ejemplo, combates contra enemigos. Estos combates se desarrollan por turnos, en los cuales el jugador puede utilizar las cartas de su inventario para obtener distintas bonificaciones.

Para implementar las distintas mecánicas del juego, se utilizaron las estructuras de datos estudiadas en la asignatura, con el fin de asignarles funciones específicas y lograr un funcionamiento eficiente del código. Para conseguir esto, pusimos en práctica conocimientos como el uso de punteros, memoria dinámica, estructuras y programación modular en C.

1.1. Objetivos

Objetivo general: Desarrollar un juego de rol (RPG) en lenguaje C, inspirado en Greed Island de Hunter × Hunter, con el objetivo de aplicar los conocimientos adquiridos en la asignatura de Estructuras de Datos, mediante la implementación de listas enlazadas, pilas y colas para gestionar las distintas mecánicas del juego.

Objetivos específicos:

- Implementar una lista doblemente enlazada para gestionar el inventario de cartas, permitiendo agregar, buscar, utilizar y descartar cartas.
- Implementar una pila para almacenar el historial de acciones realizadas por el jugador, permitiendo deshacer el último descarte.
- Utilizar una cola circular para gestionar los enemigos durante los combates, respetando el orden en el que fueron generados.
- Desarrollar un sistema de exploración que permita al jugador obtener cartas y enfrentarse a enemigos mediante eventos aleatorios.
- Permitir guardar y cargar el progreso de la partida mediante un archivo de texto.

2. Marco teórico

Listas enlazadas: Una lista enlazada es una estructura de datos dinámica compuesta por nodos, donde cada nodo almacena información y uno o más punteros que permiten establecer conexiones con otros nodos. A diferencia de los arreglos, sus elementos no necesitan encontrarse en posiciones consecutivas de memoria.

Existen cuatro tipos principales de listas enlazadas:

- **Lista enlazada simple:** Cada nodo contiene un puntero al siguiente, permitiendo recorrer la lista en una sola dirección.
- **Lista doblemente enlazada:** Cada nodo contiene dos punteros, uno al nodo siguiente y otro al anterior, permitiendo recorrer la lista en ambas direcciones.

- **Lista circular simple:** Es similar a una lista simple, pero el último nodo apunta al primero, formando una estructura circular.
- **Lista circular doble:** Combina las características de una lista doble y una circular, permitiendo recorrer sus elementos en ambas direcciones de forma circular.

Entre las principales operaciones de una lista enlazada se encuentran la inserción, eliminación, búsqueda y recorrido de nodos, permitiendo almacenar información de manera dinámica.

Pilas: Una pila es una estructura de datos que funciona bajo el principio LIFO *(Last In, First Out)*, es decir, el último elemento que se agrega es el primero en salir.

Las principales operaciones de una pila son *push*, el cual agrega un elemento a la pila, *pop*, que elimina el último elemento agregado, y *top*, que permite consultar el elemento que se encuentra en la cima sin eliminarlo. Esta estructura es útil para almacenar información cuando se necesita acceder primero a los elementos más recientes.

Colas: Una cola es una estructura de datos que funciona bajo el principio FIFO *(First In, First Out)*, lo que significa que el primer elemento que entra es el primero en salir, a diferencia de una pila.

Sus principales operaciones son encolar (*enqueue*), que permite agregar elementos al final de la cola, desencolar (*dequeue*), que elimina el elemento que se encuentra al principio, y *front*, que consulta el primer elemento sin eliminarlo. Una cola circular implementada sobre un arreglo reutiliza las posiciones liberadas avanzando los índices de forma modular, sin desplazar elementos.

Punteros y memoria dinámica: Los punteros son variables que almacenan direcciones de memoria, permitiendo acceder a los datos que se encuentran en dichas direcciones. En C son muy utilizados para trabajar con estructuras de datos, como las listas enlazadas, donde los punteros permiten conectar los distintos nodos.

Por otra parte, la memoria dinámica permite reservar memoria durante la ejecución del programa, utilizando funciones como `malloc`. Esta memoria debe ser liberada con `free` cuando ya no se necesite, para evitar problemas como las fugas de memoria.

2.1. Manejo de errores

Para evitar errores durante la ejecución del programa, se realizaron distintas comprobaciones, principalmente en las funciones del inventario, el historial y la cola. Por ejemplo, se verifica que los punteros no sean `NULL` antes de utilizarlos, evitando acceder a memoria que no corresponde. También se consideran casos como intentar seleccionar una carta cuando el inventario está vacío o ingresar un número que no corresponde a ninguna carta, mostrando un mensaje de error al usuario.

Por otra parte, se comprueba que la memoria se haya reservado correctamente mediante `malloc`. Además, se utiliza `free` para liberar la memoria de los nodos que son eliminados del inventario. También se verifica que el historial exista y que se pueda registrar una acción antes de descartar una carta. En la sección 3.5 se resume el tratamiento de cada caso límite por estructura.

3. Metodología

3.1. Organización y desarrollo del proyecto

El proyecto se organizó de forma modular: cada funcionalidad del enunciado quedó en su propio par de archivos `.h/.c`, con los prototipos y su documentación en `inc/` y la implementación en `src/`. Esto permitió que cada integrante trabajara sobre módulos distintos y que las estructuras de datos quedaran separadas de la lógica del juego.

Cuadro 1: Módulos del proyecto y estructura de datos que utilizan.

Módulo	Responsabilidad
`inventario`	Lista doblemente enlazada con centinela: agregar, eliminar, buscar, descartar y restaurar cartas.
`historial`	Pila enlazada de acciones y ventana de reversión del último descarte.
`cola`	TDA cola circular de enemigos (`s_Enemigo`).
`combate`	Genera el grupo de enemigos (cola), ejecuta los turnos y el uso de cartas en combate.
`exploracion`, `eventos`	Caminata paso a paso y eventos aleatorios (nada, combate, carta, trampa).
`carta`, `enemigo`, `personaje`	Datos y generación aleatoria de cartas, enemigos y personajes.
`partida`	Guardado y carga del estado completo en archivo.
`ui`, `comunes`, `main`	Menús, validación de entradas y bucle principal.

Todo el estado de la partida viaja en una sola estructura, `s_GameState`, que contiene el personaje, el puntero al inventario, el puntero al historial y la bandera `ejecutando`. De esta forma se evita el uso de variables globales: cada función recibe el estado que necesita por parámetro.

Trabajo en equipo y control de versiones. El proyecto se alojó en GitHub y se trabajó de forma modular para mantener separadas las funcionalidades. Los commits reflejan el avance por módulo: primero las estructuras base (`carta`, `personaje`, `enemigo`), después las estructuras de datos (inventario, historial, cola), luego la lógica del juego (exploración, combate, eventos) y finalmente el guardado/carga y la interfaz. la coordinacion se trabajo de forma presencial o online para que no hayan conflictos graves de merge.

3.2. Implementación y elección de las estructuras de datos

3.2.1. Lista doblemente enlazada con nodo centinela: inventario

El inventario de cartas se implementó con una lista doblemente enlazada, ya que durante el juego el jugador puede obtener, utilizar y descartar cartas constantemente. Esta estructura permite agregar y eliminar elementos de forma dinámica, sin establecer un tamaño máximo como ocurriría con un arreglo, y sin tener que desplazar elementos al eliminar uno del medio.

¿Por qué doble y no simple? En una lista simple, eliminar un nodo conocido exige recorrer la lista desde el inicio para encontrar su predecesor. Como cada nodo guarda `prev` y `next`, `eliminar_carta` enlaza directamente el anterior con el siguiente, dada la posición del nodo. Esta operación es la más frecuente (usar, descartar y restaurar cartas) y por eso justifica el puntero adicional.

¿Por qué centinela? La lista parte con un nodo cabecera que no guarda ninguna carta. Con él, insertar o eliminar la primera carta se trata igual que cualquier otra (el nodo previo siempre existe) y no hace falta tratar el inicio como caso especial. La lista vacía es simplemente `I->next == NULL`. Además, `eliminar_carta` rechaza explícitamente el centinela para no destruirlo por error.

```
typedef struct s_Inventario {
    s_Carta carta;            /* nombre, tipo, valor, id */
    struct s_Inventario *next;
    struct s_Inventario *prev;
} s_Inventario;
```

La lista no es circular: el último nodo tiene `next == NULL` y el centinela tiene `prev == NULL`. Por simplicidad, las nuevas cartas se insertan inmediatamente después del centinela (inserción al inicio).

Propiedad de la memoria. Cada nodo es dueño de su propia copia del nombre de la carta (`agregar_carta` duplica el texto con `malloc`) y la libera al ser eliminado (`eliminar_carta` y `vaciar_inventario`). Así, el nombre nunca depende de una cadena externa que pueda desaparecer.

Cuadro 2: Operaciones del inventario.

Operación	Descripción
`crear_inventario`	Reserva el centinela; si ya existe, solo lo vacía.
`agregar_carta`	Inserta tras una posición dada y actualiza los cuatro enlaces afectados.
`eliminar_carta`	Desenlaza el nodo y libera nombre y nodo.
`encontrar_carta`	Busca por número de posición (base 1).
`buscar_carta_nombre`	Busca por nombre recorriendo la lista con `strcmp`.
`buscar_carta_id`	Busca por id único recorriendo la lista.
`imprimir_cartas`, `cantidad_cartas`	Recorrido completo.
`vaciar_inventario`	Libera todas las cartas y deja el centinela.
`descartar_carta`	Calcula la posición, registra en el historial y elimina.
`restaurar_carta`	Reinserta la carta en su posición original (deshacer).

En el juego, la lista participa en: recoger cartas durante la exploración (`evento_looteo`), listar y buscar cartas (menú de inventario), usar cartas dentro y fuera del combate (`usar_carta_en_combate`), descartar cartas y deshacer el descarte.

3.2.2. Pila enlazada: historial de acciones

El historial se implementó como una pila (LIFO) enlazada: la cima es el puntero `nodo` de la estructura `s_historial`, y cada nodo apunta con `siguiente` a la acción inmediatamente más antigua.

¿Por qué una pila? Los requisitos del historial son exactamente los de una pila: (1) mostrar las acciones de la más reciente a la más antigua, lo que equivale a recorrer desde la cima; y (2) deshacer *únicamente la última acción*, que es lo que está en la cima. No se necesita acceder a posiciones intermedias ni al extremo antiguo, por lo que una lista doble o una cola aportarían complejidad sin beneficio. Se eligió la versión enlazada y no la de arreglo porque el historial crece con el tiempo de juego sin un tamaño conocido de antemano, y así no se requiere reubicar

memoria ni fijar un máximo. El historial sí se guarda y carga, pero la información de reversión del descarte (la carta y su posición original) no se persiste. Por ello, los descartes de sesiones anteriores permanecen como registros y no pueden deshacerse en una partida recién cargada; la reversión solo está disponible para un descarte registrado durante la sesión actual mientras siga siendo la última acción.

```
struct s_NodoHistorial {
    ContHistorial datos;               /* descripcion (char *) + tipo */
    struct s_NodoHistorial *siguiente; /* hacia el mas antiguo */
};
struct s_historial {
    struct s_NodoHistorial *nodo;      /* cima */
    int  CantidadRegistros;
    bool HayReversionPendiente;
    s_Reversion reversion;             /* carta + posicion para deshacer */
};
```

Operaciones y respeto del TDA. La pila solo se modifica por la cima:

Cuadro 3: Operaciones del historial (pila).

Operación	Descripción
`historial_make_empty`	Crea la pila vacía (`nodo = NULL`, contador en 0).
`historial_push`	Copia la descripción y apila un nodo nuevo en la cima.
`historial_pop`	Desapila la cima y libera su descripción y su nodo.
`historial_top`	Consulta la cima sin retirarla.
`historial_is_empty`	Verifica contador y puntero a la vez (detecta estados incoherentes).
`historial_imprimir`	Recorre desde la cima: de la más reciente a la más antigua.
`historial_destroy/historial_free`	Libera todos los nodos, descripciones y el propio historial.

Para el usuario del módulo, el recorrido se hace solo con `historial_top` y `historial_siguiente`; ningún otro módulo accede a la memoria de los nodos de forma directa para insertar o eliminar en el medio.

Registro de acciones con `snprintf`. Las descripciones del historial son mensajes genéricos que incluyen nombres variables ("Se derrotó al enemigo ...", "Se encontró ...", "El personaje usó ..."). Para construirlos se usa un búfer local de 128 caracteres y `snprintf` en lugar de `sprintf`, que recibe el tamaño del búfer y trunca si el texto no cabe, evitando desbordamientos. También se protege el caso de nombre `NULL` con un texto alternativo.

```
char descripcion[128];
snprintf(descripcion, sizeof(descripcion), "Se derroto al enemigo %s",
         enemigo_actual->nombre ? enemigo_actual->nombre : "desconocido");
historial_registrar(gs->Historial, descripcion, A_REGISTRO, NULL);
```

El historial luego hace su propia copia dinámica del texto (`copiar_cadena`), así que el búfer local puede desaparecer sin problema al terminar la función. En el uso de cartas en combate, el texto se arma *antes* de eliminar la carta, porque al eliminarla se libera su nombre.

Tipos de acción. Cada nodo guarda un tipo (`A_CONSUMIBLE`, `A_REGISTRO`, `A_COMBATE_I`, `A_COM_TURNO`, `A_FIN_COMBATE`, `A_DESCARTE`). Solo `A_DESCARTE` es reversible, tal como pide el

enunciado.

Deshacer el descarte. El enunciado exige que solo se pueda deshacer la última acción si fue descartar un objeto. Esto se implementó así:

1. Al descartar, `descartar_carta` calcula la posición de la carta en el inventario y guarda una copia de ella (con una copia propia del nombre, pues el nodo original se libera) en una estructura `s_Reversion`.
2. `historial_registrar` apila la acción y deja esa reversión pendiente dentro del historial (`HayReversionPendiente = true`). Si la acción es de otro tipo, cualquier reversión anterior se libera y se invalida: de este modo la ventana de deshacer es siempre de *una sola acción*.
3. `restaurar_carta` comprueba con `historial_puede_deshacer` que la cima sea un `A_DESCARTE` con reversión pendiente; si no, informa al jugador que no se puede deshacer y no modifica nada.
4. Si es posible, reinserta la carta en su posición original (`posicion_para_restaurar`; si el inventario ya no tiene suficientes cartas, la agrega al final) y recién entonces llama a `historial_deshacer`, que hace el `pop` de la cima.

Dueño de la memoria. El nombre duplicado de la carta descartada pertenece al historial desde que se registra el descarte con éxito, y se libera cuando la reversión se consume, se invalida o el historial se destruye. Si el registro falla, `descartar_carta` libera la copia y no elimina la carta, de modo que no se pierde ni se filtra memoria.

3.2.3. Cola circular: orden de enemigos en combate

Cuando el jugador se encuentra con enemigos, se genera un grupo de entre 1 y `MAX_GRUPO` (3) enemigos. A cada uno se le asigna un número de aparición (`id` = 1, 2, 3...) y se encolan en ese orden. El combate siempre se libra contra el enemigo del frente; al derrotarlo se desencola y pasa el siguiente.

¿Por qué una cola? El orden de enfrentamiento del enunciado (primero el enemigo 1, luego el 2, luego el 3) sigue la regla de primero en entrar, primero en salir; una pila invertiría el orden y una lista general permitiría saltarse enemigos, contradiciendo la regla.

¿Por qué un arreglo circular? La cantidad de enemigos del grupo se conoce al crear la cola, por lo que se reserva un único bloque de memoria con ese tamaño. Los índices `frente` y `final` avanzan de forma modular (`cola_siguiente_indice`), reutilizando posiciones sin desplazar ningún elemento, y encolar y desencolar son operaciones simples. Se evita además reservar y liberar un nodo por enemigo.

```
struct s_Cola {
    int capacidad;
    int frente;     /* indice del elemento que sale primero */
    int final;      /* indice del ultimo elemento insertado */
    int tamanho;    /* cantidad de elementos */
    s_Enemigo *arreglo;
};
```

Se mantiene un contador `tamanho` para distinguir sin ambigüedad una cola llena de una vacía (con solo `frente` y `final` ambas situaciones se verían iguales).

La cola solo se manipula por sus extremos (encolar al final, desencolar y consultar al frente) y

Cuadro 4: Operaciones de la cola.

Operación	Descripción
`cola_crear`	Reserva la estructura y el arreglo; valida capacidad ≥ 1.
`cola_encolar`	Inserta al final; falla si la cola está llena.
`cola_desencolar`	Elimina el frente; falla si está vacía.
`cola_frente`	Devuelve un puntero al frente (permite actualizar su vida en combate); `NULL` si está vacía.
`cola_esta_vacia`, `cola_esta_llena`, `cola_tamanho`	Consultas de estado.
`cola_vaciar`, `cola_destruir`	Reinicia los índices / libera el arreglo y la estructura.

su tipo es opaco (`typedef struct s_Cola *Cola`), por lo que el resto del programa no puede romper el principio FIFO accediendo al arreglo directamente. El número de enemigos en espera se calcula con el tamaño de `grupo` menos uno, usando la consulta `cola_tamanho`.

3.3. Funcionamiento del juego

Exploración. El personaje camina paso a paso (`explorar_hasta_evento`). Cada `PASOS_POR_EVENTO` (3) pasos se genera un evento con `definir_evento`, con probabilidades de 40 % combate, 20 % hallazgo de carta, 10 % trampa (pérdida de vida) y 30 % sin evento. La caminata termina cuando se encuentra una carta o un grupo de enemigos; luego se pregunta si se desea seguir explorando.

Combate por turnos. `combate` crea el grupo (cola), registra el inicio, y repite: mostrar estado, elegir entre atacar, ver inventario o usar carta. El daño es máx$(1,\ \text{ataque} - \text{defensa})$, con probabilidad de golpe crítico que lo duplica. Si el enemigo cae, se registra en el historial y se desencola; si no, contraataca. El combate termina cuando la cola queda vacía o la vida del personaje llega a 0 (fin de la partida).

Uso de cartas. Las cartas son de tipo ataque, defensa, curación, vida máxima o especial. Al usarse, modifican las estadísticas del personaje y se eliminan del inventario, y la acción se registra como `A_CONSUMIBLE`.

3.4. Guardado y carga de partida

`partida_guardar` escribe en un archivo de texto el personaje, el inventario y el historial, y `partida_cargar` los reconstruye al iniciar el juego. El formato usa una línea por elemento con un identificador al inicio:

```
PERSONAJE;nombre;vida;vidaMax;ataque;defensa
INVENTARIO;cantidad
CARTA;nombre;tipo;valor;id        (una por carta)
HISTORIAL;cantidad
ACCION;tipo;descripcion           (una por accion)
```

Las estructuras participan activamente: el inventario se reconstruye insertando cartas con `agregar_carta` y el historial con `historial_push`. Como la pila guarda la acción más reciente

en la cima, para escribirla del más antiguo al más reciente se usa una función recursiva (`escribir_historial_rec`) que primero desciende hasta el último nodo y luego escribe al volver de la recursión. Así, al cargar, cada `push` deja el elemento nuevo en la cima y el orden original se conserva. La carga valida el formato general y las cantidades; el tipo de carta se lee directamente del archivo. Ante un archivo ausente o con formato general inválido, libera lo reservado hasta ese punto y devuelve error sin modificar la partida actual.

3.5. Casos límite y manejo de memoria

Cuadro 5: Tratamiento de casos límite por estructura.

	Inventario (lista)	Historial (pila)	Combate (cola)
Vacía	`I->next == NULL`: se avisa "inventario vacío" en vez de pedir una carta.	`pop` y `top` devuelven error/`NULL`; `imprimir` muestra "Historial vacío".	`cola_desencolar` falla y `cola_frente` devuelve `NULL`.
Un elemento	El centinela permite eliminar la única carta sin caso especial; `prev`/`next` quedan consistentes.	Al hacer `pop` de la única acción, `nodo` pasa a `NULL` y el contador a 0.	Con un solo enemigo, "enemigos en espera" es 0; al desencolarlo el combate termina.
Memoria insuficiente	`malloc` revisado en nodo y copia del nombre; si falla, no se inserta y se avisa.	`malloc` revisado en la descripción y el nodo; si falla, se libera lo ya reservado y se devuelve -1.	`cola_crear` libera la estructura si falla el arreglo; `generar_grupo` destruye la cola si falla algo.
Eliminación	Se actualizan `prev`/`next`, se libera nombre y nodo; se rechaza eliminar el centinela.	`pop` libera descripción y nodo, e invalida la reversión.	`desencolar` solo avanza `frente`; el espacio se reutiliza al encolar.

Además se validan las entradas del usuario (`leer_entero_en_rango`, `leer_si_no`) y se comprueba que los punteros recibidos no sean `NULL` al inicio de cada función. Cada estructura tiene su función de liberación total (`vaciar_inventario`, `historial_free`, `cola_destruir`) para no dejar fugas de memoria al terminar un combate o la partida.

3.6. Declaración de IA

- **Lukas Bustamante:** usé ChatGPT como apoyo para resolver dudas puntuales sobre punteros y memoria dinámica, y como revisor del código del inventario en busca de fugas de memoria. El diseño y la implementación del módulo son míos; la IA se usó para discutir alternativas y confirmar que la liberación de memoria estaba bien.
- **Francisco Alvarado:** usé DeepSeek para dudas puntuales sobre manejo de archivos y modularización, y como revisor de lógica y de fugas de memoria en `partida.c` y en

el historial. El código fue escrito y comprendido por mí; la IA se usó para contrastar decisiones y detectar errores difíciles de ver.

- **Daniel Uribe:** usé Claude para entender mejor el funcionamiento de una cola circular y como ayuda para encontrar errores, tanto en el código como en el uso de Git. También la usé para revisar la presentación de la interfaz (`ui.c`/`ui.h`). La implementación de `cola.c` y la integración al combate son mías.

3.7. Contribución por estudiante

Cuadro 6: Contribución de cada integrante.

Integrante	Código	Informe	Presentación
Francisco Alvarado	Historial (pila), deshacer descarte, guardado y carga de partida (`historial.c`, `partida.c`).	Secciones 3.2.2 y 3.4; declaración de IA.	Diapositivas de historial, guardado/carga y cierre.
Lukas Bustamante	Inventario (lista doblemente enlazada): agregar, eliminar, buscar, descartar (`inventario.c`).	Secciones 3.2.1 y 4; tabla de casos límite.	Diapositivas de inventario y lista enlazada.
Daniel Uribe	Cola circular de enemigos (`cola.c`/`cola.h`), integración al combate, UI (`ui.c`).	Secciones 2 y 3.2.3; marco teórico.	Diapositivas de cola, combate e interfaz.

3.8. Procedimiento

1. Se definieron las estructuras base (`s_Personaje`, `s_Carta`, `s_Enemigo`) y el estado global `s_GameState`.
2. Se implementó el inventario con lista doblemente enlazada y centinela, y sus operaciones básicas.
3. Se implementó la cola circular y se integró al combate para respetar el orden de aparición.
4. Se implementó el historial como pila y la ventana de reversión del descarte.
5. Se desarrolló la exploración con eventos aleatorios y el combate por turnos.
6. Se implementó el guardado y carga de partida y la validación de entradas.
7. Se probaron los casos límite (estructuras vacías, un elemento, entradas inválidas) y se revisó la liberación de memoria.

4. Resultados

A continuación se muestra salida real del juego que ilustra las funcionalidades principales. Las capturas se obtuvieron ejecutando el binario compilado con `make` en una terminal Linux.

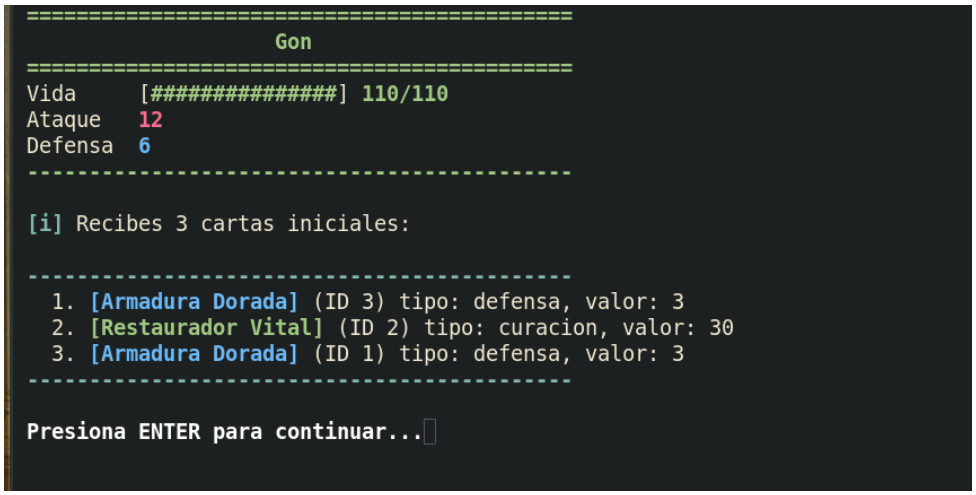


Figura 1: Estado inicial del personaje y cartas recibidas al comenzar la partida.

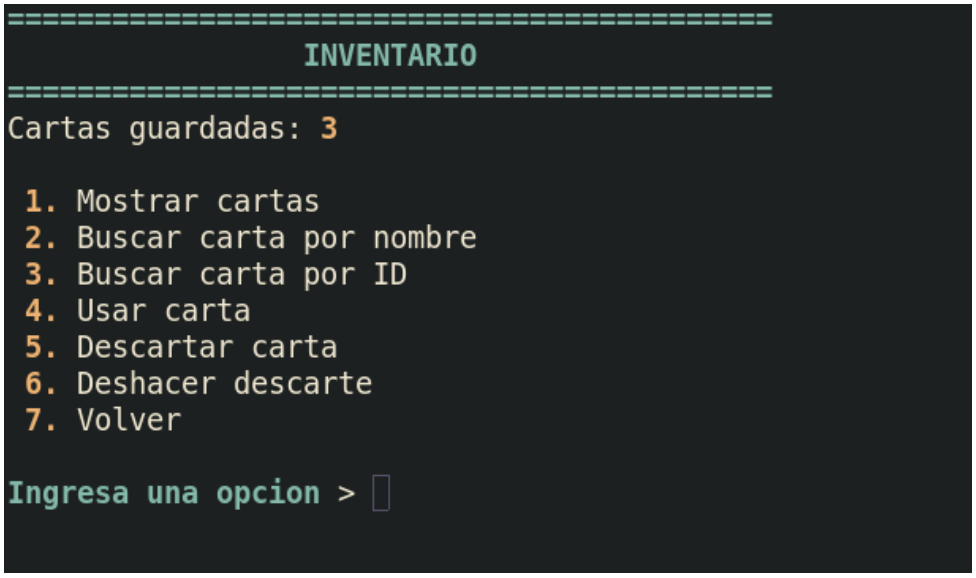


Figura 2: Menú del inventario con opciones para mostrar, buscar, usar, descartar y deshacer el descarte de cartas.

Exploración y hallazgo de carta.

```
Entras a Antokiba...

Caminas entre los arboles. No encuentras nada.
Sigues por el sendero. No encuentras nada.
Cruzas un pequeno arroyo. No encuentras nada.

[!] !Encuentras una carta en el suelo!
  [Armadura Dorada] (ID 4) tipo: defensa, valor: 3
Deseas recogerla? (s/n) > s
[OK] Armadura Dorada agregada al inventario.
```

Combate con varios enemigos (orden de aparición).



Figura 3: Combate contra un grupo de enemigos: la Bestia de Greed está al frente y el Bandido queda en espera.

Descarte y deshacer en el inventario.

```
================ INVENTARIO ================
Cartas guardadas: 3

 1. [Espada de la Verdad] (ID 1) tipo: ataque, valor: 5
 2. [Armadura Dorada]     (ID 4) tipo: defensa, valor: 3
 3. [Genio Curativo]      (ID 7) tipo: curacion, valor: 20
--------------------------------------------
Ingresa una opcion > 5

Ingrese el ID de la carta: 4
[OK] Has descartado Armadura Dorada

Ingresa una opcion > 6
[OK] Has restaurado Armadura Dorada
```

Historial (más reciente primero).

```
================ HISTORIAL ================
3 registros (del mas reciente al mas antiguo)

 1. [DESCARTE] Carta descartada
 2. [REGISTRO] Se derroto al enemigo Goblin
 3. [REGISTRO] Se recogio Armadura Dorada
===========================================
```

Guardado y carga.

```
[OK] Partida guardada en 'partida.csv'

# (reinicio del juego)
[OK] Partida cargada: Gon
```

Salida valgrind

```
Ingresa una opcion > 6

[i] Gracias por jugar. Hasta pronto!
==20169==
==20169== HEAP SUMMARY:
==20169==     in use at exit: 0 bytes in 0 blocks
==20169==   total heap usage: 11 allocs, 11 frees, 2,317 bytes allocated
==20169==
==20169== All heap blocks were freed -- no leaks are possible
==20169==
==20169== For lists of detected and suppressed errors, rerun with: -s
==20169== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Verificación con Valgrind. En una ejecución normal del juego, Valgrind no detectó fugas de memoria: al finalizar quedaron 0 bytes en uso y se liberaron las 11 asignaciones realizadas (2 317 bytes). El resumen reportó 0 errores. Este resultado corresponde a la ejecución observada.

Cuadro 7: Cumplimiento de las funcionalidades obligatorias.

Módulo	Resultado
Personaje	Creación (elección de personaje), carga y visualización del estado.
Aventura	Exploración paso a paso, hallazgo de cartas y encuentro con grupos de enemigos.
Inventario	Agregar, listar, buscar por nombre e ID, usar y eliminar (descartar) cartas.
Historial	Registro de acciones, visualización de reciente a antigua y deshacer el último descarte.
Combate	Enemigos en orden de aparición (cola), ver inventario y usar cartas.
Guardar/Cargar	Persistencia del personaje, inventario e historial en archivo.
Interfaz	Menú por terminal con validación de entradas y mensajes de error.

5. Discusión

La elección de las estructuras respondió a cómo se usa cada dato en el juego. El inventario requiere insertar y eliminar en posiciones arbitrarias, y la lista doblemente enlazada con centinela lo permite una vez conocida la posición, y buscar por nombre, por ID o por número se hace recorriendo la lista, lo que es aceptable porque el inventario es pequeño. El historial solo necesita acceder al evento más reciente, lo que calza con la pila; la política de "deshacer solo un descarte" se resolvió guardando la información de reversión junto al historial en lugar de en cada nodo, de modo que apenas se registra otra acción la reversión anterior se invalida y se libera. La cola circular modela directamente el orden de aparición de los enemigos, y al ser de tamaño conocido evita reservar un nodo por enemigo.

Una decisión importante fue la propiedad de la memoria: el nodo del inventario y el historial hacen cada uno su propia copia de los textos, y la cadena de la reversión se traspasa al historial

una vez registrada la acción. Esto simplifica la liberación, pero exige cuidado para no liberar dos veces ni dejar cadenas sin dueño.

Limitaciones.

1. La reversión pendiente no se guarda en el archivo de partida: la ventana de deshacer vive solo dentro de la sesión actual. Tras cargar una partida, la primera acción nueva que se registre invalida cualquier reversión anterior, y no es posible deshacer un descarte hecho en la sesión previa.
2. El historial no impone un tamaño máximo (la constante `HISTORIAL_MAX` está definida, pero la pila crece mientras haya memoria).
3. El inventario no tiene límite durante la partida, pero la carga desde archivo acepta hasta 1000 elementos por sección (`CARGA_MAX_ITEMS`).
4. La búsqueda de cartas recorre la lista completa.

Como mejoras futuras se podrían imponer esos límites, persistir la reversión en el archivo y ligar la dificultad a cada zona de exploración.

6. Conclusión

Durante el desarrollo de este proyecto pudimos aplicar los conocimientos adquiridos en la asignatura de Estructuras de Datos mediante la creación de un juego RPG en lenguaje C. Para ello, utilizamos listas doblemente enlazadas, pilas y colas, asignando a cada una una función específica dentro del juego.

Cada una de estas estructuras cumplió un papel importante. La lista doblemente enlazada nos permitió gestionar el inventario de cartas, la pila se utilizó para almacenar el historial de acciones y la cola circular para organizar el orden de los enemigos durante los combates. De esta manera, pudimos comprender mejor cómo se pueden utilizar estas estructuras en un programa real.

Además, el proyecto nos permitió reforzar nuestros conocimientos sobre el uso de punteros y memoria dinámica, especialmente al trabajar con la creación y eliminación de nodos, el traspaso de la propiedad de cadenas entre estructuras y el manejo de errores ante memoria insuficiente. También aprendimos a relacionar las distintas partes del programa para que funcionaran en conjunto.

En conclusión, este trabajo nos permitió llevar a la práctica los contenidos estudiados en clases y comprender la importancia de elegir correctamente las estructuras de datos según las necesidades de un programa. También nos ayudó a ganar más experiencia programando en C y a entender mejor cómo funcionan estas estructuras.